

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**

_____o0o_____



BÀI TẬP LỚN NHẬP MÔN HỌC SÂU

Tên đề tài: Phân loại bình luận độc hại

**Nhóm lớp : 02
Số thứ tự nhóm: 20**

Vũ Công Tuấn Dương MSV: B22DCKH024

HÀ NỘI, 06/2025

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Mục tiêu và phạm vi đề tài.....	1
1.2 Bố cục báo cáo	1
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	2
2.1 Xử lý dữ liệu văn bản	2
2.2 Các mô hình học sâu.....	3
2.2.1 Long Short-Term Memory (LSTM)	3
2.2.2 Bidirectional LSTM (BiLSTM)	4
2.2.3 Gated Recurrent Unit (GRU).....	4
2.2.4 Cơ chế Attention	5
2.2.5 Attention BiLSTM	6
2.2.6 Recurrent Convolutional Neural Network (RCNN).....	7
2.2.7 Transformer Encoder.....	8
2.3 Các phương pháp tối ưu hóa huấn luyện.....	10
2.3.1 Xử lý dữ liệu mất cân bằng với Class-Weighted Loss	10
2.3.2 Masked Pooling	11
2.3.3 Focal Loss.....	11
2.3.4 Label Smoothing.....	11
2.3.5 Gradient Accumulation.....	12
2.3.6 AdamW Optimizer với Weight Decay	12
2.3.7 Gradient Clipping	13
2.3.8 OneCycleLR.....	13
2.3.9 Early Stopping	14
2.3.10 Tìm kiếm ngưỡng tối ưu.....	15

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ.....	16
3.1 Tập dữ liệu	16
3.2 Cấu hình thử nghiệm.....	17
3.3 So sánh tổng quan các mô hình	18
3.4 Kết quả chi tiết từng mô hình	19
3.4.1 Kết quả của RCNN.....	19
3.4.2 Kết quả của Attention BiLSTM	20
3.4.3 Kết quả của BiLSTM.....	20
3.4.4 Kết quả của Transformer.....	21
3.5 Phân tích ảnh hưởng của các kỹ thuật tối ưu hóa	21
3.6 Nhận xét chung	23
3.7 Kết quả sau tối ưu hóa và đánh giá chéo.....	24
3.7.1 Kết quả trên Jigsaw sau tối ưu hóa	24
3.7.2 Kết quả trên HateXplain	25
3.7.3 So sánh trước và sau tối ưu hóa.....	25
3.7.4 Thảo luận về hiệu suất của Transformer	26
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	27
TÀI LIỆU THAM KHẢO.....	31

DANH MỤC BẢNG BIỂU

Bảng 3.1	So sánh hiệu suất các mô hình trên tập kiểm định (159,571 mẫu)	18
Bảng 3.2	So sánh kết quả trên Jigsaw trước và sau tối ưu hóa	25
Bảng 3.3	Kết quả trên HateXplain — 3 mô hình	25
Bảng 3.4	Tổng hợp mức cải thiện F1 Macro	25

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Mục tiêu và phạm vi đề tài

Mục tiêu của đề tài là nghiên cứu, cài đặt từ gốc (from scratch) và so sánh các mô hình Deep Learning cho bài toán phân loại bình luận độc hại, nhằm tìm hiểu sâu về kiến trúc và hành vi của từng mô hình. Việc cài đặt từ đầu giúp hiểu rõ cơ chế hoạt động bên trong thay vì chỉ sử dụng các thư viện có sẵn.

Về phạm vi nghiên cứu, đề tài tập trung vào việc cài đặt và so sánh ba mô hình Deep Learning chính, bao gồm: Recurrent Convolutional Neural Network (RCNN) với phiên bản cải tiến EnhancedRCNN [1], Attention BiLSTM [2], và Transformer Encoder [3]. Tất cả các mô hình này đều được cài đặt từ đầu mà không sử dụng các thư viện có sẵn như nn.LSTM, nn.GRU, hay nn.Transformer của PyTorch, nhằm mục đích thể hiện sự hiểu biết sâu về kiến trúc bên trong của các mô hình. Việc cài đặt từ gốc bao gồm xây dựng LSTM Cell và GRU Cell từ các phép toán ma trận cơ bản, cơ chế Attention từ đầu, các lớp Self-Attention và Multi-Head Attention, cũng như các chiến lược Pooling có trọng số (masked mean/max pooling).

Đề tài cũng áp dụng nhiều kỹ thuật tối ưu hóa: Masked Pooling, Focal Loss [4], Label Smoothing [5], Gradient Accumulation, AdamW Optimizer [6], Gradient Clipping [7], OneCycleLR Scheduler [8], Early Stopping dựa trên ROC-AUC, và kỹ thuật tìm kiếm Optimal Thresholds.

Để thực hiện các thử nghiệm, đề tài sử dụng hai bộ dữ liệu: Jigsaw Toxic Comment Classification Challenge [9] và HateXplain [10]. Chi tiết về các bộ dữ liệu được trình bày trong Chương 3.

1.2 Bố cục báo cáo

Báo cáo được tổ chức thành bốn chương. Chương 1 giới thiệu tổng quan về mục tiêu và phạm vi đề tài. Chương 2 trình bày cơ sở lý thuyết về các mô hình và kỹ thuật tối ưu hóa được sử dụng. Chương 3 mô tả các thử nghiệm, kết quả đánh giá và phân tích. Chương 4 tổng kết kết quả đạt được và đề xuất hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Xử lý dữ liệu văn bản

Việc xử lý dữ liệu văn bản đóng vai trò nền tảng trong bất kỳ bài toán xử lý ngôn ngữ tự nhiên nào, bao gồm cả bài toán phân loại bình luận độc hại. Theo cuốn sách "Modern Information Retrieval" của Baeza-Yates và Ribeiro-Neto (1999), tiền xử lý văn bản là bước quan trọng để chuẩn hóa dữ liệu đầu vào và loại bỏ nhiễu, giúp cải thiện đáng kể chất lượng của các mô hình phân loại [11].

Quá trình tách từ bắt đầu bằng việc chuyển đổi văn bản gốc thành danh sách các token IDs, đồng thời tạo ra attention mask để đánh dấu các token thực sự so với các token đệm. Mỗi câu được thêm các token đặc biệt là [CLS] (classification) ở đầu và [SEP] (separator) ở cuối để phân tách các câu khi cần thiết. Độ dài tối đa của chuỗi được giới hạn ở 128 token, giúp cân bằng giữa khả năng nắm bắt ngữ cảnh và tốc độ huấn luyện. Attention mask sẽ có giá trị 1 cho các token thực và 0 cho các token đệm, cho phép mô hình bỏ qua phần đệm khi tính toán.

Trong bài toán phân loại đa nhãn, mỗi bình luận được gán nhãn bởi 6 cột nhị phân tương ứng với 6 loại độc hại: toxic, severe_toxic, obscene, threat, insult, và identity_hate. Điều đáng chú ý là một bình luận có thể đồng thời thuộc nhiều nhãn. Điều này phân biệt bài toán phân loại đa nhãn với phân loại đa lớp, đòi hỏi việc sử dụng Binary Cross-Entropy loss thay vì Categorical Cross-Entropy [12]. Hàm mất mát BCEWithLogitsLoss được tính như sau:

$$\mathcal{L}_{BCE} = -\frac{1}{N \times C} \sum_{i=1}^N \sum_{c=1}^C [y_{ic} \log(\sigma(x_{ic})) + (1 - y_{ic}) \log(1 - \sigma(x_{ic}))] \quad (2.1)$$

Trong đó N là số mẫu, C là số lớp (6), y_{ic} là nhãn thực của mẫu i lớp c , x_{ic} là log-its đầu ra của mô hình, và σ là hàm sigmoid. Khi sử dụng trọng số lớp (pos_weight), công thức trở thành:

$$\mathcal{L}_{BCE}^{weighted} = -\frac{1}{N \times C} \sum_{i=1}^N \sum_{c=1}^C w_c [y_{ic} \log(\sigma(x_{ic})) + (1 - y_{ic}) \log(1 - \sigma(x_{ic}))] \quad (2.2)$$

Với w_c là trọng số của lớp c , được tính bằng tỷ lệ $\frac{\text{số mẫu âm}_c}{\text{số mẫu dương}_c}$.

2.2 Các mô hình học sâu

2.2.1 Long Short-Term Memory (LSTM)

Mạng Long Short-Term Memory (LSTM) là một biến thể của mạng RNN (Recurrent Neural Network) được giới thiệu bởi Hochreiter và Schmidhuber vào năm 1997, nhằm giải quyết hai vấn đề cơ bản của RNN truyền thống: tiêu biến gradient và bùng nổ gradient [13]. Khi chuỗi đầu vào quá dài, gradient trong quá trình lan truyền ngược qua thời gian (BPTT) có xu hướng giảm rất nhanh hoặc tăng rất nhanh, khiến mô hình không thể học được các phụ thuộc từ xa trong chuỗi.

Kiến trúc cốt lõi của LSTM dựa trên khái niệm cell state (trạng thái ô), đóng vai trò như một "băng truyền" cho phép thông tin chảy dọc theo chuỗi mà hầu như không bị thay đổi. Cell state được điều khiển bởi ba cổng: Forget Gate, Input Gate, và Output Gate.

Các công thức toán học của LSTM được biểu diễn như sau:

$$f_t = \sigma(W_{hf}h_{t-1} + W_{xf}x_t + b_f) \quad (\text{Forget Gate}) \quad (2.3)$$

Forget Gate quyết định thông tin nào từ cell state trước đó cần được loại bỏ. Hàm sigmoid σ đưa ra giá trị trong khoảng $[0,1]$, trong đó 0 có nghĩa là "loại bỏ hoàn toàn" và 1 có nghĩa là "giữ lại hoàn toàn".

$$i_t = \sigma(W_{hi}h_{t-1} + W_{xi}x_t + b_i) \quad (\text{Input Gate}) \quad (2.4)$$

Input Gate quyết định thông tin mới nào sẽ được thêm vào cell state.

$$\tilde{c}_t = \tanh(W_{hc}h_{t-1} + W_{xc}x_t + b_c) \quad (\text{Candidate Cell State}) \quad (2.5)$$

Candidate Cell State tạo ra các giá trị ứng viên để thêm vào cell state. Hàm tanh đưa ra giá trị trong khoảng $[-1,1]$, cho phép cả giá trị dương và âm.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{Cell State Update}) \quad (2.6)$$

Cell State mới được cập nhật bằng cách kết hợp thông tin cũ (được điều khiển bởi Forget Gate) và thông tin mới (được điều khiển bởi Input Gate). Toán tử $\odot$ là phép nhân Hadamard (theo từng phần tử).

$$o_t = \sigma(W_{ho}h_{t-1} + W_{xo}x_t + b_o) \quad (\text{Output Gate}) \quad (2.7)$$

Output Gate quyết định phần nào của cell state sẽ được xuất ra.

$$h_t = o_t \odot \tanh(c_t) \quad (\text{Hidden State}) \quad (2.8)$$

Hidden State mới là kết quả của việc nhân Output Gate với tanh của Cell State, quyết định thông tin nào được truyền ra ngoài.

2.2.2 Bidirectional LSTM (BiLSTM)

Bidirectional LSTM (BiLSTM) là một mở rộng của LSTM, cho phép xử lý chuỗi đầu vào theo cả hai hướng cùng lúc: từ trái sang phải (tiền) và từ phải sang trái (lùi) [7]. Ý tưởng cốt lõi là mỗi từ trong câu không chỉ phụ thuộc vào ngữ cảnh đứng trước nó mà còn phụ thuộc vào ngữ cảnh đứng sau nó. Ví dụ trong câu "I don't hate this movie", từ "hate" mang nghĩa tiêu cực, nhưng trong ngữ cảnh đầy đủ "don't hate" lại mang nghĩa tích cực.

Về mặt kỹ thuật, BiLSTM bao gồm hai mạng LSTM chạy song song. Công thức tính hidden state của BiLSTM tại mỗi vị trí t :

$$\vec{h}_t = \text{LSTM}_{\rightarrow}(x_t, \vec{h}_{t-1}) \quad (\text{Forward}) \quad (2.9)$$

$$\overleftarrow{h}_t = \text{LSTM}_{\leftarrow}(x_t, \overleftarrow{h}_{t+1}) \quad (\text{Backward}) \quad (2.10)$$

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (\text{Concatenation}) \quad (2.11)$$

Hidden state của BiLSTM được tạo bằng cách nối hidden state từ LSTM hướng tiến và hidden state từ LSTM hướng lùi tại vị trí đó. Do đó, kích thước hidden state của BiLSTM gấp đôi so với LSTM thông thường: $\text{hidden_dim}_{BiLSTM} = 2 \times \text{hidden_dim}_{LSTM}$.

Pooling cuối cùng sử dụng Global Average Pooling để tổng hợp thông tin từ tất cả các bước thời gian. Với BiLSTM, mỗi hidden state đã có chiều gấp đôi, nên vector đầu ra sau pooling có kích thước $\text{hidden_size} \times 2$.

2.2.3 Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU) là một biến thể khác của RNN, được đề xuất bởi Cho vào năm 2014 trong bài báo về RNN Encoder-Decoder cho dịch máy thống kê [14], với thiết kế đơn giản hóa hơn so với LSTM. GRU sử dụng ít cổng hơn (chỉ 2 cổng thay vì 3) và không sử dụng cell state riêng biệt, chỉ dùng hidden state duy

nhất, giúp giảm số lượng tham số cần học khoảng 25%.

Các công thức toán học của GRU:

$$z_t = \sigma(W_{hz}h_{t-1} + W_{xz}x_t + b_z) \quad (\text{Update Gate}) \quad (2.12)$$

Update Gate z_t kết hợp vai trò của Forget Gate và Input Gate trong LSTM. Khi z_t gần 1, mô hình giữ lại hầu hết thông tin cũ; khi z_t gần 0, mô hình thêm nhiều thông tin mới.

$$r_t = \sigma(W_{hr}h_{t-1} + W_{xr}x_t + b_r) \quad (\text{Reset Gate}) \quad (2.13)$$

Reset Gate r_t kiểm soát mức độ thông tin quá khứ cần được bỏ qua khi tính toán hidden state ứng viên mới. Nếu r_t gần 0, mô hình bỏ qua hoàn toàn hidden state trước đó.

$$\tilde{h}_t = \tanh(W_{hn}(r_t \odot h_{t-1}) + W_{xn}x_t + b_n) \quad (\text{Candidate Hidden State}) \quad (2.14)$$

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1} \quad (\text{Hidden State Update}) \quad (2.15)$$

Hidden state mới là sự nội suy giữa trạng thái ứng viên và hidden state trước đó, được điều khiển bởi update gate. Công thức này cho phép mô hình linh hoạt quyết định bao nhiêu thông tin cũ cần được giữ lại.

Ưu điểm chính của GRU so với LSTM là ít tham số hơn, dẫn đến huấn luyện nhanh hơn và ít nguy cơ quá khớp hơn. Trong nhiều thử nghiệm thực tế, hiệu suất của GRU và LSTM thường tương đương nhau, và sự lựa chọn giữa hai kiến trúc phụ thuộc vào đặc điểm cụ thể của bài toán và dữ liệu.

2.2.4 Cơ chế Attention

Cơ chế Attention (chú ý) là một trong những đột phá quan trọng nhất trong lĩnh vực học sâu, được giới thiệu ban đầu trong bài toán dịch máy nơ-ron bởi Bahdanau và cộng sự vào năm 2014 [15]. Ý tưởng cốt lõi của Attention là thay vì dựa hoàn toàn vào hidden state cuối cùng để đại diện cho toàn bộ chuỗi (như trong RNN truyền thống), mô hình có thể "chú ý" (tập trung) vào các phần khác nhau của chuỗi đầu vào với mức độ khác nhau.

Trong bài toán phân loại bình luận độc hại, không phải từ nào cũng quan trọng

như nhau. Ví dụ trong câu "You are an idiot", từ "idiot" mang ý nghĩa quan trọng nhất để xác định bình luận là toxic, trong khi các từ như "are", "an" gần như không cung cấp thông tin hữu ích.

Công thức toán học của Self-Attention được sử dụng trong đề tài:

$$e_t = \tanh(W_a h_t + b_a) \quad (\text{Energy Score}) \quad (2.16)$$

Bước đầu tiên tính các điểm năng lượng e_t cho mỗi bước thời gian, trong đó h_t là hidden state tại bước thời gian t và W_a là ma trận trọng số có thể huấn luyện. Điểm năng lượng đại diện cho mức độ "quan trọng" của mỗi bước thời gian.

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)} = \text{softmax}(e_t) \quad (\text{Attention Weights}) \quad (2.17)$$

Attention weights α_t được tính bằng softmax của các điểm năng lượng, đảm bảo tổng các trọng số bằng 1 ($\sum_{t=1}^T \alpha_t = 1$). Điều này cho phép tính giải thích được: ta có thể biết được mô hình đang tập trung vào phần nào của văn bản.

$$c = \sum_{t=1}^T \alpha_t h_t \quad (\text{Context Vector}) \quad (2.18)$$

Context vector c là tổng có trọng số của tất cả hidden states. Context vector này được sử dụng thay cho mean pooling hoặc hidden state cuối cùng làm đầu vào cho lớp phân loại. Điểm khác biệt chính so với mean pooling ($\bar{h} = \frac{1}{T} \sum h_t$) là attention cho phép mô hình học để tập trung vào các phần quan trọng.

Yang và cộng sự (2016) đã phát triển kiến trúc Hierarchical Attention Networks sử dụng attention ở nhiều cấp độ và đạt kết quả ấn tượng trên bài toán phân loại văn bản [2].

2.2.5 Attention BiLSTM

Attention BiLSTM là sự kết hợp giữa kiến trúc BiLSTM và cơ chế Attention, tận dụng ưu điểm của cả hai. BiLSTM cung cấp khả năng nắm bắt ngữ cảnh hai chiều, trong khi Attention cho phép mô hình tập trung vào các từ quan trọng nhất khi đưa ra quyết định phân loại.

Trong mô hình này, đầu ra của BiLSTM là một chuỗi các hidden states (mỗi hidden state có chiều gấp đôi do nối hai hướng), sau đó Self-Attention được áp dụng lên chuỗi này để tạo ra một context vector duy nhất đại diện cho toàn bộ câu. Attention weights được tính dựa trên các hidden states từ BiLSTM, cho phép mô

hình học được tổ hợp ngữ cảnh hai chiều của từng từ trước khi quyết định mức độ quan trọng.

2.2.6 Recurrent Convolutional Neural Network (RCNN)

RCNN (Recurrent Convolutional Neural Network) là kiến trúc kết hợp giữa RNN và CNN, được đề xuất bởi Lai và cộng sự vào năm 2015 cho bài toán phân loại văn bản tại hội nghị AAAI [1]. Ý tưởng cốt lõi của RCNN là sử dụng BiLSTM để thu thập ngữ cảnh hai chiều cho mỗi từ, sau đó kết hợp ngữ cảnh đó với embedding gốc của từ để tạo ra biểu diễn phong phú hơn, rồi cuối cùng sử dụng Max Pooling thay vì Average Pooling để trích xuất các đặc trưng quan trọng nhất.

Kiến trúc RCNN trong đề tài bao gồm các bước sau:

Bước 1: BiLSTM tạo ngữ cảnh hai chiều

$$c_L(t) = \overrightarrow{h_t} \quad (\text{Ngữ cảnh trái tại vị trí } t) \quad (2.19)$$

$$c_R(t) = \overleftarrow{h_t} \quad (\text{Ngữ cảnh phải tại vị trí } t) \quad (2.20)$$

Bước 2: Nối ngữ cảnh với embedding

$$x_t^{combined} = [c_L(t); e_t; c_R(t)] \quad (\text{Biểu diễn kết hợp}) \quad (2.21)$$

Vector biểu diễn tại mỗi vị trí được tạo bằng cách nối ngữ cảnh trái, embedding gốc, và ngữ cảnh phải. Kích thước của vector này là: $\dim(c_L) + \dim(e_t) + \dim(c_R) = 2 \times \text{hidden_size} + \text{embedding_dim}$.

Bước 3: Tầng Fusion với tanh

$$y_t = \tanh(W_f \cdot x_t^{combined} + b_f) \quad (\text{Fusion}) \quad (2.22)$$

Tầng Fusion tích hợp các đặc trưng từ ba nguồn (ngữ cảnh trái, embedding, ngữ cảnh phải) thành một vector biểu diễn duy nhất cho mỗi vị trí.

Bước 4: Max Pooling

$$\hat{y} = \max_{t \in [1, T]} y_t \quad (\text{Global Max Pooling}) \quad (2.23)$$

Global Max Pooling lấy giá trị lớn nhất theo chiều thời gian, giúp tập trung vào những đặc trưng nổi bật nhất.

2.2.7 Transformer Encoder

Transformer là kiến trúc được giới thiệu trong bài báo "Attention Is All You Need" của Vaswani và cộng sự vào năm 2017 tại hội nghị NeurIPS, đánh dấu một bước tiến lớn trong lĩnh vực xử lý ngôn ngữ tự nhiên [3]. Điểm khác biệt cốt lõi của Transformer so với các mô hình RNN là nó không sử dụng cơ chế tuần tự, mà thay vào đó xử lý toàn bộ chuỗi đầu vào song song trong một bước duy nhất.

Do Transformer không xử lý tuần tự nên nó không có thông tin về vị trí của các từ trong câu một cách tự nhiên. Positional Encoding được sử dụng để bổ sung thông tin vị trí:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (\text{Vị trí chẵn}) \quad (2.24)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (\text{Vị trí lẻ}) \quad (2.25)$$

Trong đó pos là vị trí của từ, i là chiều của embedding, và d là tổng chiều của embedding. Các hàm sine và cosine với các tần số khác nhau tạo ra một vector duy nhất cho mỗi vị trí mà mô hình có thể học được.

Self-Attention là cơ chế cốt lõi của Transformer. Công thức attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.26)$$

Trong đó Q (Query), K (Key), và V (Value) là ba ma trận được tính từ đầu vào: $Q = XW_Q$, $K = XW_K$, $V = XW_V$. Hệ số $\sqrt{d_k}$ trong mẫu số giúp ổn định gradient trong quá trình huấn luyện khi d_k lớn.

Multi-Head Attention chạy nhiều đầu attention song song:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.27)$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i) \quad (2.28)$$

Mỗi head với một không gian con khác nhau, cho phép mô hình học các loại quan hệ khác nhau đồng thời: head này có thể học quan hệ ngữ pháp, head kia học quan hệ ngữ nghĩa, và một head khác học các mẫu phủ định.

Transformer Encoder Block bao gồm hai thành phần chính với Residual Connection và Layer Normalization:

$$x' = \text{LayerNorm}(x + \text{MultiHead}(x, x, x)) \quad (2.29)$$

$$x'' = \text{LayerNorm}(x' + \text{FFN}(x')) \quad (2.30)$$

Trong đó Feed-Forward Network (FFN) gồm hai lớp tuyến tính với hàm kích hoạt ReLU:

$$\text{FFN}(x) = W_2 \cdot \text{ReLU}(W_1 \cdot x + b_1) + b_2 \quad (2.31)$$

Layer Normalization [16] được tính như sau:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (2.32)$$

Trong đó $\mu = \frac{1}{d} \sum_{i=1}^d x_i$ và $\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$ lần lượt là trung bình và phương sai của đầu vào. Các tham số γ và β là các tham số có thể học được.

Pre-Layer Normalization (Pre-LN): Khác với Post-LN được mô tả ở trên (LayerNorm sau residual), các phiên bản cải tiến của Transformer sử dụng Pre-LN, áp dụng LayerNorm trước sublayer:

$$x' = x + \text{MultiHead}(\text{LayerNorm}(x), \text{LayerNorm}(x), \text{LayerNorm}(x)) \quad (2.33)$$

$$x'' = x' + \text{FFN}(\text{LayerNorm}(x')) \quad (2.34)$$

Xiong và cộng sự (2020) [17] đã chứng minh Pre-LN vượt trội so với Post-LN, đặc biệt với các Transformer nhiều tầng. Pre-LN giúp gradient flow ổn định hơn trong quá trình lan truyền ngược, tránh bùng nổ gradient ở các tầng gần đầu vào.

CLS Token: Thay vì sử dụng mean pooling trên tất cả các đầu ra, phiên bản cải tiến thêm một token đặc biệt [CLS] vào đầu chuỗi. Token này có khả năng học để tổng hợp thông tin toàn bộ câu thông qua cơ chế self-attention, và đầu ra tại vị trí này được sử dụng trực tiếp cho phân loại. Phương pháp này được sử dụng phổ biến trong BERT [18] và các mô hình dựa trên Transformer khác.

Trong đề tài này, Transformer được cấu hình với Pre-LN, 4 tầng mã hóa, CLS token, và đầu ra tại vị trí CLS được dùng làm biểu diễn cho phân loại.

2.3 Các phương pháp tối ưu hóa huấn luyện

2.3.1 Xử lý dữ liệu mất cân bằng với Class-Weighted Loss

Một trong những thách thức phổ biến trong các bài toán phân loại là sự mất cân bằng giữa các lớp. Khi số lượng mẫu của các lớp chênh lệch lớn, mô hình có xu hướng thiên về lớp đa số và bỏ qua lớp thiểu số.

Để giải quyết vấn đề này, đề tài sử dụng Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss) với trọng số lớp (pos_weight). Trọng số cho mỗi lớp được tính bằng:

$$w_c = \frac{\text{số mẫu âm}_c}{\text{số mẫu dương}_c} = \frac{N - p_c}{p_c} \quad (2.35)$$

Trong đó p_c là số mẫu dương tính của lớp c và N là tổng số mẫu. Trọng số này nhân với loss của các mẫu dương, khiến việc bỏ sót một mẫu độc hại có chi phí cao hơn nhiều so với việc dự đoán sai một mẫu không độc hại.

Để tránh trọng số quá lớn gây ra việc phạt quá mức lớp đa số, đề tài áp dụng clipping:

$$w_c^{clipped} = \min(w_c, \text{max_weight}) \quad (2.36)$$

Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng [4]:

$$\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (2.37)$$

Trong đó p_t là xác suất của lớp đúng, α_t là trọng số cân bằng, và γ là tham số tập trung. Focal Loss giảm trọng số của các mẫu dễ phân loại và tập trung vào các mẫu khó.

Sokolova và Lapalme (2009) đã phân tích chi tiết các độ đo hiệu suất cho bài toán phân loại và nhận thấy rằng F1-score đặc biệt hữu ích khi dữ liệu mất cân bằng [19]:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.38)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (2.39)$$

F1 Macro là trung bình F1 của các lớp: $F1_{macro} = \frac{1}{C} \sum_{c=1}^C F1_c$.

2.3.2 Masked Pooling

Pooling là bước quan trọng để tổng hợp thông tin từ các bước thời gian của chuỗi thành một vector đại diện duy nhất. Tuy nhiên, khi sử dụng đệm để đồng nhất độ dài chuỗi trong batch, các token đệm (giá trị 0) sẽ làm "pha loãng" biểu diễn cuối cùng nếu pooling được tính trên toàn bộ độ dài.

Trong các phiên bản đầu, cả mean pooling và max pooling đều tính trên toàn bộ chuỗi, bao gồm cả các token đệm. Điều này dẫn đến việc các chuỗi ngắn có đến 50% token là đệm, làm giảm đáng kể chất lượng biểu diễn. Để khắc phục, đề tài sử dụng masked pooling, tận dụng attention_mask (mảng nhị phân với 1 cho token thực và 0 cho token đệm):

Masked Mean Pooling:

$$\bar{h} = \frac{\sum_{t=1}^T m_t \cdot h_t}{\sum_{t=1}^T m_t + \epsilon} \quad (2.40)$$

Masked Max Pooling:

$$\tilde{h}_i = \max_{t:m_t=1} h_{t,i} \quad (2.41)$$

Trong đó m_t là giá trị attention mask tại bước thời gian t , h_t là hidden state của RNN. Với max pooling, các token đệm được gán $-\infty$ trước khi lấy max, đảm bảo chúng không ảnh hưởng đến kết quả.

2.3.3 Focal Loss

Focal Loss [4] là một cải tiến của Binary Cross-Entropy, được thiết kế để xử lý dữ liệu mất cân bằng bằng cách giảm trọng số của các mẫu dễ phân loại:

$$\mathcal{L}_{focal} = -\frac{1}{N} \sum_{i=1}^N (1 - p_{t,i})^\gamma \log(p_{t,i}) \quad (2.42)$$

Với $\gamma = 2$, một mẫu có $p_t = 0.9$ (dễ) sẽ có trọng số $(1 - 0.9)^2 = 0.01$, trong khi mẫu có $p_t = 0.1$ (khó) có trọng số $(1 - 0.1)^2 = 0.81$. Điều này giúp mô hình tập trung vào các mẫu khó (thường là các lớp thiểu số) và cải thiện F1 Macro trên các lớp này.

2.3.4 Label Smoothing

Label Smoothing là kỹ thuật regularization được đề xuất bởi Szegedy và cộng sự (2016) [5], nhằm giảm quá khớp và cải thiện hiệu chỉnh của mô hình. Thay vì sử dụng nhãn one-hot cứng ($y_c \in \{0, 1\}$), label smoothing tạo ra nhãn mềm:

$$y_c^{smooth} = y_c \cdot (1 - \epsilon) + \frac{\epsilon}{C} \quad (2.43)$$

Với $\epsilon = 0.1$, nhân 1 trở thành $1 - \epsilon + \epsilon/C$ và nhân 0 trở thành ϵ/C . Điều này ngăn mô hình trở nên quá tự tin và cải thiện khả năng tổng quát hóa.

2.3.5 Gradient Accumulation

Gradient Accumulation cho phép mô phỏng kích thước batch hiệu quả lớn hơn khi bộ nhớ GPU hạn chế. Thay vì cập nhật tham số sau mỗi batch nhỏ (micro-batch), gradient được tích lũy qua k batch trước khi thực hiện một bước tối ưu:

$$g_{accum} = \frac{1}{k} \sum_{i=1}^k \nabla_{\theta} \mathcal{L}_i \quad (2.44)$$

$$\theta \leftarrow \theta - \eta \cdot \text{AdamW}(\theta, g_{accum}) \quad (2.45)$$

Với $k = 4$ và micro-batch size = 64, kích thước batch hiệu quả đạt 256, mang lại gradient ổn định hơn và cải thiện chất lượng hội tụ. Smith và cộng sự (2018) [8] đã chứng minh rằng kích thước batch hiệu quả lớn giúp huấn luyện ổn định và có thể đạt được hiệu suất tương đương với kích thước batch thực tế.

2.3.6 AdamW Optimizer với Weight Decay

Adam (Adaptive Moment Estimation) được giới thiệu bởi Kingma và Ba năm 2015 [20], kết hợp Momentum và RMSprop:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{First moment estimate}) \quad (2.46)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{Second moment estimate}) \quad (2.47)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{Bias correction}) \quad (2.48)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (\text{Parameter update}) \quad (2.49)$$

AdamW (Adam with Decoupled Weight Decay) được đề xuất bởi Loshchilov và Hutter năm 2017 [6], tách biệt weight decay khỏi gradient update:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \cdot \lambda \cdot \theta_t \quad (2.50)$$

Trong đó λ là hệ số weight decay (10^{-4} trong đề tài). Việc tách biệt weight decay giúp đảm bảo regularization effect nhất quán bất kể giá trị của các thành phần thích

nghi.

Weight decay (L2 regularization) đóng vai trò quan trọng trong việc ngăn chặn quá khớp. Nó khuyến khích các weights có giá trị nhỏ, giúp cải thiện khả năng tổng quát hóa của mô hình [12].

Srivastava và cộng sự (2014) đã nghiên cứu về Dropout như một phương pháp regularization hiệu quả khác [21]. Trong đề tài, dropout với tỷ lệ 0.3 được áp dụng trước lớp fully connected cuối cùng để ngăn quá khớp.

2.3.7 Gradient Clipping

Gradient Clipping là kỹ thuật quan trọng đặc biệt đối với các mô hình RNN. Trong quá trình lan truyền ngược qua thời gian, gradient được nhân qua nhiều bước thời gian liên tiếp, có thể dẫn đến bùng nổ gradient.

Kỹ thuật Gradient Clipping giới hạn giá trị của gradient bằng cách điều chỉnh tỷ lệ nếu norm vượt quá ngưỡng:

$$\text{if } \|g\|_2 > \text{max_norm} : \quad g \leftarrow g \cdot \frac{\text{max_norm}}{\|g\|_2} \quad (2.51)$$

Trong đó $\|g\|_2 = \sqrt{\sum_i g_i^2}$ là L2 norm của gradient. Trong đề tài, max_norm = 1.0 được sử dụng. Gradient clipping được áp dụng sau khi gọi loss.backward() và trước khi gọi optimizer.step().

Graves và cộng sự (2012) đã áp dụng gradient clipping thành công trong các mạng RNN cho nhận dạng giọng nói và nhận thấy rằng kỹ thuật này đặc biệt quan trọng khi huấn luyện trên các chuỗi dài [7].

LeCun và cộng sự (1998) đã thảo luận chi tiết về các thủ thuật huấn luyện mạng nơ-ron, bao gồm tầm quan trọng của việc kiểm soát gradient để đảm bảo quá trình huấn luyện ổn định [22].

2.3.8 OneCycleLR

OneCycleLR được đề xuất bởi Smith và Topin (2018) [8], là một chiến lược điều chỉnh tốc độ học dựa trên nguyên lý "hội tụ siêu tốc". Thay vì warmup rồi giảm dần, OneCycleLR thực hiện đúng một chu kỳ: tăng tốc độ học từ lr_{min} lên lr_{max} trong giai đoạn đầu, sau đó giảm về lr_{min} trong giai đoạn cuối.

$$lr(t) = \begin{cases} lr_{min} + (lr_{max} - lr_{min}) \cdot \frac{t}{T_{warmup}}, & 0 \leq t < T_{warmup} \\ lr_{max} - (lr_{max} - lr_{min}) \cdot \left[1 - \frac{1}{2} \left(1 + \cos \left(\pi \cdot \frac{t - T_{warmup}}{T_{total} - T_{warmup}} \right) \right) \right], & T_{warmup} \leq t < T_{total} \end{cases} \quad (2.52)$$

Trong cấu hình đề tài: $lr_{max} = 1 \times 10^{-3}$, $pct_{start} = 0.1$ (warmup chiếm 10% tổng số bước), và T_{total} là tổng số bước batch. Khác với Warmup Cosine Scheduler giảm dần đến 0, OneCycleLR chỉ có đúng một chu kỳ tăng-giảm duy nhất trong toàn bộ quá trình huấn luyện.

Smith (2018) [8] đã chứng minh rằng OneCycleLR giúp mô hình hội tụ nhanh hơn đáng kể so với các bộ lập lịch truyền thống, có thể đạt được độ chính xác tương đương chỉ với một vài epoch. Nguyên lý hoạt động dựa trên việc cho phép mô hình "thoát khỏi" các cực tiểu địa phương hẹp ở giai đoạn đầu với tốc độ học cao, sau đó "hạ cánh" vào một cực tiểu địa phương rộng và phẳng hơn ở giai đoạn cuối với tốc độ học thấp.

2.3.9 Early Stopping

Early Stopping là kỹ thuật regularization đơn giản nhưng hiệu quả, nhằm ngăn chặn quá khớp bằng cách dừng quá trình huấn luyện khi độ đo kiểm định không còn cải thiện.

Đề tài sử dụng Early Stopping với cấu hình:

- **patience** = 2 epoch
- **mode** = 'max' (theo dõi ROC-AUC)
- **Độ đo** = ROC-AUC (thay vì loss)

ROC-AUC (Receiver Operating Characteristic - Area Under Curve) đo lường khả năng phân biệt của mô hình giữa các lớp dương tính và âm tính:

$$AUC = \int_0^1 TPR(t) dFPR(t) \quad (2.53)$$

Trong đó TPR (True Positive Rate) = Recall và FPR (False Positive Rate) = $\frac{FP}{FP+TN}$. AUC bằng 0.5 có nghĩa là dự đoán ngẫu nhiên, và AUC bằng 1.0 có nghĩa là phân loại hoàn hảo [23].

Lý do sử dụng ROC-AUC thay vì loss là vì loss có thể bị ảnh hưởng bởi trọng

số lớp, trong khi ROC-AUC đo lường khả năng phân biệt thực sự của mô hình một cách độc lập với ngưỡng. Raschka (2018) đã thảo luận chi tiết về tầm quan trọng của việc sử dụng các độ đo phù hợp để tránh quá khớp [24].

```

Input: patience  $p$ , best_score, current_score
if current_score > best_score then
    best_score  $\leftarrow$  current_score
    patience_counter  $\leftarrow$  0
    Lưu model checkpoint
else
    patience_counter  $\leftarrow$  patience_counter + 1
    if patience_counter  $\geq p$  then
        Khôi phục model từ checkpoint tốt nhất
        Dừng huấn luyện
    
```

Algorithm 1: Thuật toán Early Stopping

Wolpert (1997) đã chứng minh No Free Lunch Theorems, cho thấy rằng không có thuật toán tối ưu nhất cho mọi bài toán [25]. Early Stopping là một ví dụ về việc áp dụng nguyên tắc này trong thực tế.

2.3.10 Tìm kiếm ngưỡng tối ưu

Trong phân loại đa nhãn với đầu ra là xác suất từ hàm sigmoid, ngưỡng mặc định để chuyển đổi xác suất thành nhãn nhị phân thường là 0.5. Tuy nhiên, với dữ liệu mất cân bằng, ngưỡng 0.5 không phải lúc nào cũng tối ưu.

Đề tài áp dụng kỹ thuật tìm kiếm ngưỡng tối ưu riêng biệt cho từng lớp:

$$\theta_c^* = \arg \max_{\theta \in [0.1, 0.9]} F1_c(\theta) \quad (2.54)$$

Với mỗi lớp c , duyệt qua các giá trị ngưỡng từ 0.1 đến 0.9 với bước nhảy 0.01 và chọn ngưỡng tối ưu dựa trên F1-score cao nhất. Phương pháp này cho phép mỗi lớp có một ngưỡng riêng phù hợp với phân phối và đặc điểm của lớp đó.

Fawcett (2006) đã thảo luận chi tiết về tầm quan trọng của việc lựa chọn ngưỡng phù hợp trong các bài toán phân loại và cách nó ảnh hưởng đến đường cong ROC và các độ đo hiệu suất khác nhau [23].

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

3.1 Tập dữ liệu

Bộ dữ liệu được sử dụng trong đề tài là Jigsaw Toxic Comment Classification Challenge từ Kaggle [9] với mục tiêu phát triển các mô hình có khả năng nhận diện các loại bình luận độc hại khác nhau trên Wikipedia. Bộ dữ liệu này bao gồm các bình luận từ Wikipedia đã được gán nhãn thủ công bởi các chuyên gia kiểm duyệt theo 6 loại độc hại khác nhau. Tổng cộng có 159,571 bình luận trong tập dữ liệu, được chia theo tỷ lệ 80% cho tập huấn luyện và 20% cho tập kiểm định, sử dụng phương pháp chia tách phân tầng để đảm bảo phân phối các lớp được giữ nguyên trong cả hai tập.

Phân tích chi tiết phân bố nhãn cho thấy mức độ mất cân bằng nghiêm trọng của dữ liệu. Lớp toxic có khoảng 15,000 mẫu dương tính (chiếm khoảng 9.4% tổng số mẫu), lớp obscene có khoảng 8,000 mẫu (5.0%), và lớp insult có khoảng 7,600 mẫu (4.8%). Các lớp thiểu số đặc biệt nghiêm trọng: severe_toxic chỉ có khoảng 1,600 mẫu (1.0%), identity_hate có khoảng 1,400 mẫu (0.9%), và đặc biệt threat chỉ có khoảng 320 mẫu (0.2%) - một con số rất nhỏ so với tổng thể 159,571 bình luận. Sự mất cân bằng này đặt ra thách thức lớn cho việc huấn luyện mô hình, buộc phải áp dụng các kỹ thuật xử lý mất cân bằng như Class-Weighted Loss để đảm bảo mô hình không bỏ qua hoàn toàn các lớp thiểu số. Hiện tượng này cũng đã được nghiên cứu trong nhiều bài toán phân loại văn bản khác, như đã được báo cáo bởi các nghiên cứu trước đây về phát hiện lời lẽ thù hận [26].

Tập kiểm định chứa 31,915 bình luận, được sử dụng để đánh giá hiệu suất của mô hình và thực hiện các thử nghiệm như tìm kiếm các ngưỡng tối ưu. Tập kiểm thử chứa khoảng 153,000 bình luận không có nhãn, được sử dụng để tạo bài nộp cho cuộc thi Kaggle. Việc đánh giá trên tập kiểm định thay vì tập kiểm thử (không có nhãn thực) là cách tiếp cận tiêu chuẩn trong nghiên cứu, cho phép so sánh và phân tích chi tiết hiệu suất của từng mô hình. Raschka (2018) đã thảo luận chi tiết về tầm quan trọng của việc sử dụng tập kiểm định để đánh giá và lựa chọn mô hình, tránh để tập kiểm thử ảnh hưởng đến quá trình phát triển mô hình [24].

Bên cạnh Jigsaw, đề tài sử dụng bộ dữ liệu HateXplain [10] để đánh giá chéo. HateXplain [10] bao gồm 20,109 bình luận được thu thập từ Twitter và Reddit, được gán nhãn bởi ba người gán nhãn độc lập với ba nhãn: normal (bình thường), offensive (xúc phạm), và hatespeech (lời thù hận). Khác với Jigsaw (đa nhãn, 6 nhãn độc lập), HateXplain là bài toán đa lớp với một nhãn duy nhất cho mỗi bình luận. Dữ liệu được chia theo tỷ lệ 80% cho huấn luyện (16,087 mẫu) và 20% cho

kiểm định (4,022 mẫu). Việc sử dụng hai bộ dữ liệu với nguồn gốc khác nhau (Wikipedia so với Twitter/Reddit) và cấu trúc nhãn khác nhau (đa nhãn so với đa lớp) cho phép đánh giá tính tổng quát của các kiến trúc mô hình trên nhiều điều kiện dữ liệu.

3.2 Cấu hình thử nghiệm

Để đảm bảo tính công bằng trong việc so sánh giữa các mô hình, tất cả các mô hình đều được huấn luyện với cùng bộ siêu tham số và chỉ khác nhau về kiến trúc mô hình. Các siêu tham số chính bao gồm: kích thước batch = 64, tốc độ học = 2×10^{-3} , số epoch tối đa = 10 với Early Stopping (patience = 2), độ dài tối đa chuỗi = 128 token, chiều embedding = 128, chiều hidden state = 256, số tầng RNN = 2, và tỷ lệ dropout = 0.3. Ngoài ra, các tham số tối ưu hóa bao gồm weight decay = 10^{-4} , tỷ lệ warmup = 0.1, và giới hạn gradient clip = 1.0. Việc sử dụng chung siêu tham số cho tất cả các mô hình là một phương pháp phổ biến trong nghiên cứu so sánh các kiến trúc mô hình, như được áp dụng trong nhiều nghiên cứu trước đây [1], [3].

Tokenizer được sử dụng là AutoTokenizer từ thư viện Hugging Face Transformers [27] với mô hình đã được huấn luyện trước "distilbert-base-uncased". DistilBERT là phiên bản nhỏ gọn hơn và nhanh hơn BERT, được huấn luyện trên 6GB dữ liệu văn bản tiếng Anh, do đó tokenizer của nó có khả năng tách từ thông minh, hiểu được subword tokenization (tách từ thành các tiền tố và hậu tố) giúp xử lý tốt các từ hiếm gặp và từ không có trong từ điển. Sanh và cộng sự (2019) đã chứng minh rằng DistilBERT đạt 97% hiệu suất của BERT trên bộ đánh giá GLUE trong khi chỉ sử dụng 60% tham số và thời gian suy luận nhanh hơn 60% [28].

Cấu hình chi tiết cho từng mô hình như sau: LSTM và Attention LSTM sử dụng 2 tầng LSTM với hidden size 256, embedding dimension 128; BiLSTM và Attention BiLSTM sử dụng 2 tầng LSTM forward và 2 tầng LSTM backward với hidden size 256 mỗi hướng; GRU sử dụng 2 tầng GRU với hidden size 256; RCNN sử dụng 2 tầng BiLSTM với hidden size 256 mỗi hướng, tầng fusion với hàm kích hoạt tanh; Transformer sử dụng 2 tầng mã hóa, 8 đầu attention, feed-forward dimension 512. Tất cả các mô hình đều sử dụng dropout 0.3 trước lớp fully connected cuối cùng.

Song song với thử nghiệm gốc, đề tài tiến hành thử nghiệm bổ sung với cấu hình tối ưu hóa trên cả hai bộ dữ liệu. Cấu hình mới sử dụng batch size = 32, tốc độ học = 1×10^{-3} , gradient accumulation với 2 bước (batch size hiệu quả = 64), số epoch tối đa = 5, và OneCycleLR scheduler. Về kiến trúc, Transformer được nâng cấp lên 4 tầng mã hóa với Pre-LN và CLS token thay vì Post-LN và mean pooling, RCNN được mở rộng thêm một lớp fc_hidden với hàm kích hoạt ReLU sau max pooling

(EnhancedRCNN), và Attention BiLSTM giữ nguyên kiến trúc self-attention gốc. Masked pooling, label smoothing ($\epsilon = 0.1$), và focal loss ($\gamma = 2.0$) cũng được áp dụng.

3.3 So sánh tổng quan các mô hình

Kết quả huấn luyện và đánh giá trên tập kiểm định cho thấy sự khác biệt đáng kể giữa các mô hình. Bảng tổng hợp dưới đây trình bày các chỉ số hiệu suất chính của tất cả 7 mô hình đã được thử nghiệm. Các độ đo được sử dụng bao gồm ROC-AUC (diện tích dưới đường cong ROC), F1 Macro (trung bình F1 của các lớp), và F1 Micro (F1 tính tổng hợp trên tất cả các mẫu và nhãn), được tính toán dựa trên các công thức đã được phân tích chi tiết bởi Sokolova và Lapalme (2009) trong nghiên cứu về các độ đo cho bài toán phân loại [19].

Mô hình	Tham số (M)	Best AUC	Final AUC	F1 Macro	F1 Micro
RCNN	6.6	0.9839	0.9831	0.5689	0.6655
Attention BiLSTM	6.5	0.9834	0.9830	0.6311	0.7324
Attention LSTM	4.9	0.9812	0.9791	0.5650	0.6820
GRU	4.6	0.9805	0.9793	0.5146	0.6198
BiLSTM	6.3	0.9800	0.9776	0.5520	0.6689
Transformer	4.3	0.9789	0.9788	0.4625	0.5947
LSTM	4.8	0.9786	0.9761	0.5459	0.6658

Bảng 3.1: So sánh hiệu suất các mô hình trên tập kiểm định (159,571 mẫu)

Phân tích kết quả cho thấy RCNN đạt ROC-AUC cao nhất (0.9839), cho thấy khả năng xếp hạng các mẫu dương tính cao hơn các mẫu âm tính của mô hình này là tốt nhất. Attention BiLSTM đứng thứ hai với AUC 0.9834 nhưng đạt F1 Macro cao nhất (0.6311), cho thấy sự cân bằng tốt nhất giữa precision và recall. Sự khác biệt về độ đo này là do AUC đo khả năng phân biệt ở mọi ngưỡng, trong khi F1 Macro tập trung vào hiệu suất tại các ngưỡng tối ưu cho từng lớp. Fawcett (2006) đã giải thích chi tiết sự khác biệt giữa AUC và các độ đo phụ thuộc ngưỡng như F1-score [23].

Các mô hình không có attention (BiLSTM, LSTM, GRU) và Transformer đều có F1 Macro thấp hơn đáng kể so với các mô hình có attention (Attention BiLSTM, Attention LSTM). Cụ thể, Attention BiLSTM (0.6311) cao hơn BiLSTM (0.5520) khoảng 0.08 điểm, và Attention LSTM (0.5650) cao hơn LSTM (0.5459) khoảng 0.02 điểm. Điều này chứng minh rằng cơ chế attention thực sự mang lại lợi ích đáng kể trong bài toán phân loại bình luận độc hại, cho phép mô hình tập trung vào các từ quan trọng và cải thiện khả năng phân biệt giữa các trường hợp thực sự thuộc nhãn và các trường hợp dương tính giả. Kết quả này nhất quán với các nghiên cứu trước đây về hiệu quả của attention trong các bài toán NLP [2], [15].

Đáng chú ý, GRU với ít tham số nhất (4.6 triệu) và huấn luyện nhanh hơn các mô hình BiLSTM đạt F1 Macro thấp nhất trong nhóm RNN (0.5146), cho thấy thiết kế đơn giản hóa của GRU có thể không đủ để nắm bắt các phụ thuộc phức tạp trong bài toán này. Transformer với kiến trúc hoàn toàn khác (không có tính hồi quy) đạt F1 Macro thấp nhất (0.4625), có thể do cần nhiều dữ liệu và thời gian huấn luyện hơn để đạt hiệu suất tối ưu. Kết quả này cho thấy rằng kiến trúc phù hợp cho một bài toán cụ thể cần được lựa chọn dựa trên đặc điểm của bài toán và dữ liệu, như đã được nhấn mạnh trong nghiên cứu về "No Free Lunch Theorems" của Wolpert (1997) [25].

3.4 Kết quả chi tiết từng mô hình

3.4.1 Kết quả của RCNN

RCNN đạt ROC-AUC cao nhất trong tất cả các mô hình (0.9839) và chi tiết hiệu suất trên từng lớp cho thấy mô hình này đặc biệt xuất sắc trong việc nhận diện bình luận obscene với F1-score 0.81 (precision 0.76, recall 0.87) và bình luận threat với F1-score 0.43 (precision 0.29, recall 0.79). Điểm mạnh của RCNN nằm ở kiến trúc kết hợp ngữ cảnh hai chiều từ BiLSTM với embedding gốc, tạo ra biểu diễn phong phú cho mỗi từ, sau đó sử dụng max pooling để trích xuất các đặc trưng nổi bật nhất. Lai và cộng sự (2015) đã đề xuất kiến trúc RCNN trong bài báo tại AAAI và chứng minh hiệu quả của nó trên nhiều bài toán phân loại văn bản [1]. Cách tiếp cận này đặc biệt hiệu quả khi các từ độc hại (như "fuck", "shit" cho obscene) xuất hiện rõ ràng và max pooling có thể dễ dàng chọn ra chúng.

Tuy nhiên, RCNN cũng có hạn chế rõ rệt trên các lớp thiểu số: severe_toxic chỉ đạt precision 0.25 với recall 0.96 ($F1 = 0.40$), và identity_hate chỉ đạt precision 0.20 với recall 0.85 ($F1 = 0.32$). Precision thấp trên các lớp này cho thấy mô hình có xu hướng dự đoán quá nhiều, tạo ra nhiều dương tính giả. Nguyên nhân có thể là do max pooling chỉ chọn giá trị lớn nhất, có thể bỏ qua các tín hiệu quan trọng trong các trường hợp không có từ khóa rõ ràng. Khi áp dụng các ngưỡng tối ưu, F1 Macro tăng lên 0.6328 và F1 Micro tăng lên 0.7428, cho thấy việc tinh chỉnh ngưỡng vẫn mang lại cải thiện đáng kể.

Một điểm cần lưu ý là RCNN có số lượng tham số nhiều nhất (6.6 triệu) trong số tất cả các mô hình, đòi hỏi nhiều bộ nhớ GPU và thời gian huấn luyện lâu hơn. Việc sử dụng BiLSTM 2 tầng để tạo ngữ cảnh hai chiều cộng với tầng fusion tạo ra chi phí tính toán cao hơn so với các mô hình khác. Tuy nhiên, với ROC-AUC cao nhất, RCNN vẫn là một lựa chọn tốt cho các ứng dụng cần ưu tiên khả năng phân biệt tổng thể.

3.4.2 Kết quả của Attention BiLSTM

Attention BiLSTM đạt F1 Macro cao nhất (0.6311 với các ngưỡng tối ưu = 0.6503) và ROC-AUC thứ hai (0.9834), cho thấy đây là mô hình có sự cân bằng tốt nhất giữa các độ đo hiệu suất. Kết quả chi tiết cho thấy mô hình đạt được precision cao hơn đáng kể so với RCNN ở hầu hết các lớp, đặc biệt là toxic (0.67 so với 0.62), severe_toxic (0.29 so với 0.25), và identity_hate (0.32 so với 0.20). Đồng thời, Attention BiLSTM vẫn duy trì recall ở mức tương đối cao cho hầu hết các lớp, với recall cao nhất ở obscene (0.94) và insult (0.92). Sự kết hợp giữa BiLSTM và attention đã được nghiên cứu trước đó bởi Yang và cộng sự (2016) trong Hierarchical Attention Networks và đã chứng minh hiệu quả trên nhiều bài toán phân loại văn bản [2].

Điểm nổi bật của Attention BiLSTM là khả năng đạt F1 cao nhất trên lớp threat (0.48 với các ngưỡng tối ưu) so với tất cả các mô hình khác. Điều này cho thấy cơ chế attention giúp mô hình học được cách nhận diện các mẫu đe dọa ngay cả khi chúng không chứa từ khóa rõ ràng, bằng cách tổng hợp thông tin từ nhiều phần của câu. Tuy nhiên, recall cho threat vẫn chỉ ở mức 0.68, thấp hơn nhiều so với các lớp khác, cho thấy vẫn còn khó khăn trong việc nhận diện tất cả các bình luận threat trong tập dữ liệu. Nguyên nhân có thể là do số lượng mẫu threat quá ít (chỉ 320 mẫu), khiến mô hình khó học đầy đủ các biểu diễn cần thiết.

Số lượng tham số của Attention BiLSTM là 6.5 triệu, gần tương đương với RCNN (6.6 triệu), và thời gian huấn luyện cũng tương tự (khoảng 9 phút mỗi epoch). Tuy nhiên, với F1 Macro cao hơn và precision tốt hơn trên các lớp thiểu số, Attention BiLSTM có thể được coi là lựa chọn tốt hơn cho các ứng dụng cần cân bằng precision và recall.

3.4.3 Kết quả của BiLSTM

BiLSTM không có attention đạt ROC-AUC 0.9800 và F1 Macro 0.5520 (với các ngưỡng tối ưu = 0.5825), thấp hơn đáng kể so với Attention BiLSTM. Kết quả này xác nhận vai trò quan trọng của cơ chế attention trong việc cải thiện hiệu suất phân loại. BiLSTM sử dụng Global Average Pooling thay vì attention pooling, nghĩa là tất cả các bước thời gian được tính bình quân với trọng số bằng nhau, khiến các từ ít quan trọng (như "the", "a", "is") cũng đóng góp ngang bằng với các từ quan trọng trong biểu diễn cuối cùng. Graves và cộng sự (2012) đã nghiên cứu về việc sử dụng RNN hai chiều trong nhận dạng giọng nói và nhận thấy rằng việc xử lý hai chiều mang lại cải thiện đáng kể so với RNN một chiều [7].

Mặc dù recall trung bình của BiLSTM khá cao (0.88), precision lại rất thấp (0.36), dẫn đến F1 Macro thấp. Điều này cho thấy BiLSTM có xu hướng dự đoán

quá nhiều các trường hợp là độc hại, tạo ra nhiều dương tính giả. Cụ thể, trên lớp threat BiLSTM chỉ đạt precision 0.13, thấp hơn rất nhiều so với Attention BiLSTM (0.37), cho thấy việc thiếu cơ chế attention khiến mô hình không thể phân biệt tốt giữa các bình luận thực sự là threat và các bình luận có từ ngữ tương tự nhưng không mang ý nghĩa đe dọa. BiLSTM có 6.3 triệu tham số và huấn luyện nhanh hơn một chút so với Attention BiLSTM (do không có tầng attention), nhưng khoảng cách về hiệu suất là đáng kể.

3.4.4 Kết quả của Transformer

Transformer đạt ROC-AUC 0.9789 nhưng F1 Macro thấp nhất trong tất cả các mô hình (0.4625 với các ngưỡng tối ưu = 0.5593). Đây là kết quả đáng chú ý vì Transformer là kiến trúc hiện đại và mạnh mẽ nhất trong danh sách, được sử dụng rộng rãi trong các mô hình ngôn ngữ lớn. Vaswani và cộng sự (2017) đã giới thiệu Transformer trong bài báo "Attention Is All You Need" và chứng minh hiệu quả vượt trội trên bài toán dịch máy [3]. Tuy nhiên, kết quả trong đề tài này cho thấy Transformer cần được điều chỉnh cấu hình phù hợp để đạt hiệu suất tối ưu trên bài toán phân loại bình luận độc hại.

Nguyên nhân chính của hiệu suất thấp có thể bao gồm: thứ nhất, việc sử dụng embedding khởi tạo ngẫu nhiên thay vì các embedding đã được huấn luyện trước như GloVe [29] hoặc FastText [30] khiến mô hình phải học tất cả các biểu diễn từ đầu, đòi hỏi nhiều dữ liệu và thời gian hơn; thứ hai, Transformer thường cần huấn luyện nhiều epoch hơn (20-50 epoch) để đạt hiệu suất tối ưu, trong khi thử nghiệm chỉ chạy 10 epoch; thứ ba, cấu hình 2 tầng và 8 đầu attention có thể chưa đủ mạnh để nắm bắt các phụ thuộc phức tạp trong bài toán này. Devlin và cộng sự (2019) đã chứng minh rằng việc sử dụng BERT đã được huấn luyện trước với tinh chỉnh mang lại cải thiện vượt bậc trên nhiều bài toán NLP [18].

Transformer có ưu điểm về tốc độ huấn luyện: mỗi epoch chỉ mất khoảng 2 phút (nhanh hơn 4-5 lần so với BiLSTM) do xử lý song song toàn bộ chuỗi thay vì tuần tự. Số lượng tham số cũng ít nhất (4.3 triệu). Nếu được huấn luyện với các embedding đã được huấn luyện trước và nhiều epoch hơn, Transformer có tiềm năng đạt hiệu suất cao hơn.

3.5 Phân tích ảnh hưởng của các kỹ thuật tối ưu hóa

Các kỹ thuật tối ưu hóa đã được áp dụng trong đề tài đã đóng góp đáng kể vào việc cải thiện hiệu suất của tất cả các mô hình. So sánh với kết quả từ các thử nghiệm trước đó trên tập 10,000 mẫu, việc huấn luyện trên toàn bộ tập dữ liệu (159,571 mẫu) mang lại cải thiện vượt trội: ROC-AUC tăng từ khoảng 0.95 lên 0.98 (tăng khoảng 0.03 điểm), và F1 Macro tăng từ khoảng 0.43 lên 0.63 (tăng khoảng

0.20 điểm). Điều này chứng minh tầm quan trọng của việc huấn luyện trên dữ liệu đầy đủ để mô hình học được các biểu diện tốt. Goodfellow và cộng sự (2016) đã thảo luận trong cuốn "Deep Learning" rằng mạng nơ-ron học sâu thường cần lượng dữ liệu lớn để đạt hiệu suất tối ưu [12].

Class-Weighted Loss là kỹ thuật có ảnh hưởng lớn nhất đến hiệu suất trên các lớp thiểu số. Trước khi áp dụng trọng số lớp, mô hình gần như bỏ qua hoàn toàn các lớp như threat và identity_hate do số lượng mẫu quá ít. Sau khi áp dụng trọng số lớp với clipping ở mức 50, mô hình bắt đầu nhận diện được các mẫu từ các lớp này, với recall dao động từ 0.68 đến 0.96 tùy mô hình. Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng trong object detection và nhận thấy rằng việc giảm trọng số của các mẫu dễ phân loại giúp mô hình tập trung vào các mẫu khó [4]. Tuy nhiên, precision vẫn thấp ở các lớp thiểu số (0.11-0.37), cho thấy vẫn còn nhiều dương tính giả và cần các cải tiến thêm như tăng cường dữ liệu hoặc các embedding đã được huấn luyện trước.

AdamW với weight decay giúp ngăn quá khớp và cải thiện khả năng tổng quát hóa của mô hình. Loshchilov và Hutter (2017) đã chứng minh rằng AdamW cải thiện đáng kể so với Adam thông thường trong việc regularization và đạt kết quả tốt hơn trên các bộ đánh giá [6]. Gradient clipping đảm bảo quá trình huấn luyện ổn định, đặc biệt quan trọng đối với các mô hình RNN. Graves và cộng sự (2012) đã áp dụng gradient clipping thành công trong các mạng RNN cho nhận dạng giọng nói và nhận thấy rằng kỹ thuật này đặc biệt quan trọng khi huấn luyện trên các chuỗi dài [7]. Early Stopping đã ngăn chặn quá khớp hiệu quả, với hầu hết các mô hình dừng ở epoch 4-7 thay vì chạy đủ 10 epoch, cho thấy mô hình đã đạt đến điểm tối ưu trước khi bắt đầu quá khớp.

Warmup Cosine LR Scheduler giúp quá trình huấn luyện ổn định và hội tụ tốt hơn. Khi quan sát đường cong tốc độ học, ta thấy tốc độ học tăng tuyến tính trong giai đoạn warmup (khoảng 200 bước đầu), sau đó giảm dần theo hàm cosine trong phần còn lại. Vaswani và cộng sự (2017) đã sử dụng bộ lập lịch warmup trong huấn luyện Transformer và nhận thấy rằng nó giúp ổn định quá trình huấn luyện trong giai đoạn đầu [3]. Điều này cho phép mô hình khám phá rộng rãi không gian tham số ở giai đoạn đầu và tinh chỉnh ở giai đoạn cuối.

Kỹ thuật tìm kiếm các ngưỡng tối ưu riêng biệt cho từng lớp cũng mang lại cải thiện đáng kể. Các ngưỡng tối ưu thường cao hơn 0.5 đáng kể (từ 0.62 đến 0.89), cho thấy mô hình có xu hướng đưa ra xác suất khá cao cho cả các mẫu không độc hại. Việc tăng ngưỡng giúp lọc bớt các dương tính giả và cải thiện precision.

Fawcett (2006) đã thảo luận chi tiết về tầm quan trọng của việc lựa chọn ngưỡng phù hợp trong các bài toán phân loại và cách nó ảnh hưởng đến đường cong ROC và các độ đo hiệu suất khác nhau [23].

3.6 Nhận xét chung

Qua quá trình thử nghiệm và đánh giá 7 mô hình học sâu khác nhau trên bài toán phân loại bình luận độc hại, một số nhận xét quan trọng có thể được rút ra. Thứ nhất, cơ chế Attention thực sự mang lại lợi ích đáng kể trong bài toán này. Attention BiLSTM vượt trội hơn BiLSTM ở cả ROC-AUC lẫn F1 Macro, và sự cải thiện này đến từ khả năng tập trung vào các từ quan trọng của attention thay vì tính bình quân tất cả các bước thời gian như trong mean pooling truyền thống. Kết quả này nhất quán với các nghiên cứu trước đây của Bahdanau và cộng sự (2014) về hiệu quả của attention trong các bài toán NLP [15].

Thứ hai, kiến trúc hai chiều vượt trội hơn kiến trúc một chiều trong hầu hết các trường hợp. BiLSTM đạt kết quả tốt hơn LSTM, và Attention BiLSTM đạt kết quả tốt hơn Attention LSTM. Điều này cho thấy việc nắm bắt ngữ cảnh cả trước và sau một từ là quan trọng trong việc hiểu ý nghĩa của văn bản, đặc biệt khi có các từ phủ định hoặc các mẫu phức tạp. Graves và cộng sự (2012) đã chứng minh rằng việc xử lý hai chiều mang lại cải thiện đáng kể trong nhận dạng giọng nói [7].

Thứ ba, việc xử lý dữ liệu mất cân bằng là yếu tố then chốt quyết định thành công của bài toán. Không có Class-Weighted Loss, mô hình gần như không thể nhận diện được các lớp thiểu số như threat (chỉ 0.2% dữ liệu). Với Class-Weighted Loss và clipping, recall cho tất cả các lớp đều đạt mức cao (0.68-0.97), cho thấy mô hình đã học được cách nhận diện các mẫu từ các lớp thiểu số. Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng và nhận thấy rằng nguyên tắc giảm trọng số của các mẫu dễ phân loại mang lại cải thiện đáng kể [4].

Thứ tư, Precision vẫn là điểm yếu chung của tất cả các mô hình, đặc biệt trên các lớp thiểu số. Precision thấp nhất là ở threat (0.09-0.37) và identity_hate (0.11-0.32), cho thấy mô hình vẫn tạo ra nhiều dương tính giả trên các lớp này. Nguyên nhân có thể là do số lượng mẫu quá ít khiến mô hình không thể học được các biểu diễn đủ tốt để phân biệt chính xác giữa các trường hợp thực sự và giả của các lớp thiểu số. Sokolova và Lapalme (2009) đã phân tích chi tiết về các độ đo hiệu suất cho bài toán phân loại và nhận thấy rằng F1-score là một độ đo cân bằng, nhưng precision và recall có thể thay đổi đáng kể tùy thuộc vào ngưỡng được chọn [19].

Thứ năm, Transformer với cấu hình hiện tại chưa đạt hiệu suất tốt như các mô hình RNN trong thử nghiệm này. Tuy nhiên, điều này không có nghĩa là Trans-

former không phù hợp cho bài toán này, mà có thể cần cấu hình khác (nhiều tầng hơn, các embedding đã được huấn luyện trước, nhiều epoch huấn luyện) để phát huy hết tiềm năng của kiến trúc. Vaswani và cộng sự (2017) đã huấn luyện Transformer với nhiều tầng hơn và đạt hiệu suất vượt trội trên bài toán dịch máy [3].

Cuối cùng, sự kết hợp giữa RCNN (hai chiều + embedding fusion + max pooling) và Attention BiLSTM (hai chiều + attention) cho thấy hai hướng tiếp cận khác nhau để cải thiện biểu diễn: RCNN tập trung vào việc trích xuất các đặc trưng mạnh nhất (max pooling), trong khi Attention BiLSTM tập trung vào việc tổng hợp có trọng số các đặc trưng. Lai và cộng sự (2015) đã đề xuất kiến trúc RCNN và nhận thấy rằng việc kết hợp RNN và CNN mang lại hiệu quả tốt trên bài toán phân loại văn bản [1]. Cả hai đều vượt trội hơn các mô hình cơ bản không có các cải tiến này.

3.7 Kết quả sau tối ưu hóa và đánh giá chéo

3.7.1 Kết quả trên Jigsaw sau tối ưu hóa

Ở lần chạy đầu (Bảng 3.1), hai mô hình dựa trên RNN dẫn đầu: RCNN đạt Best AUC cao nhất (0.9839) nhưng F1 Macro chỉ 0.5689, trong khi Attention BiLSTM đạt F1 Macro tốt nhất (0.6311) với AUC 0.9834. Transformer tụt lại với F1 Macro 0.4625, thấp hơn RCNN 0.1064 điểm.

Sau khi áp dụng các kỹ thuật tối ưu hóa, cả ba mô hình đều cải thiện đáng kể về F1 Macro (Bảng 3.2), trong khi AUC thay đổi không đáng kể (± 0.0007). Mức cải thiện cụ thể:

- **RCNN:** F1 Macro tăng từ 0.5689 lên 0.6530 (+14.8%), nhiều nhất về giá trị tuyệt đối nhờ masked pooling và label smoothing cải thiện hiệu chỉnh.
- **Transformer:** F1 Macro tăng từ 0.4625 lên 0.6274 (+35.7%), ấn tượng nhất về tỷ lệ, nhờ Pre-LN, CLS token và 4 tầng mã hóa — thu hẹp khoảng cách với RCNN từ 0.1064 xuống còn 0.0256.
- **Attention BiLSTM:** F1 Macro tăng nhẹ từ 0.6311 lên 0.6344 (+0.5%), do kiến trúc gốc vốn đã hoạt động tốt.

AUC gần như không thay đổi giữa các phiên bản, cho thấy khả năng xếp hạng các mẫu vốn đã tốt từ phiên bản gốc. Các kỹ thuật tối ưu hóa chủ yếu tác động đến việc cải thiện precision và recall tại các ngưỡng quyết định cụ thể, từ đó nâng cao F1 Macro.

Mô hình	Phiên bản	AUC	F1 Macro	F1 Micro
RCNN	Gốc	0.9839	0.5689	0.6655
RCNN	Tối ưu	0.9846	0.6530	0.7703
Attention BiLSTM	Gốc	0.9834	0.6311	0.7324
Attention BiLSTM	Tối ưu	0.9833	0.6344	0.7608
Transformer	Gốc	0.9789	0.4625	0.5947
Transformer	Tối ưu	0.9797	0.6274	0.7497

Bảng 3.2: So sánh kết quả trên Jigsaw trước và sau tối ưu hóa

3.7.2 Kết quả trên HateXplain

Cả ba mô hình đều được huấn luyện trên HateXplain với cấu hình tối ưu. Khác với Jigsaw (đa nhãn, 6 nhãn), HateXplain là bài toán đa lớp với 3 nhãn nên sử dụng CrossEntropyLoss. Kết quả được trình bày trong Bảng 3.3.

Mô hình	AUC	F1 Macro	F1 Micro	Precision	Recall
RCNN	0.8156	0.6399	0.6571	0.6484	0.6414
Attention BiLSTM	0.8039	0.6169	0.6380	0.6239	0.6229
Transformer	0.8087	0.6375	0.6402	0.6415	0.6356

Bảng 3.3: Kết quả trên HateXplain — 3 mô hình

Nhìn chung, AUC trên HateXplain thấp hơn đáng kể so với Jigsaw (0.80-0.82 so với 0.98), phản ánh độ khó cao hơn của dữ liệu từ Twitter và Reddit cũng như số lượng mẫu ít hơn (20,109 so với 159,571). RCNN đạt AUC cao nhất (0.8156) và F1 Macro cao nhất (0.6399), khẳng định lại ưu thế của kiến trúc kết hợp BiLSTM với max pooling. Transformer đạt F1 Macro 0.6375, rất sát với RCNN (0.6399) và vượt trội so với Attention BiLSTM (0.6169). Đây là điểm khác biệt quan trọng so với kết quả trên Jigsaw, nơi Transformer ở phiên bản gốc có F1 Macro thấp hơn nhiều so với các mô hình còn lại.

3.7.3 So sánh trước và sau tối ưu hóa

Bảng 3.4 tổng hợp mức cải thiện của từng mô hình trên cả hai bộ dữ liệu.

Mô hình	Jigsaw (gốc)	Jigsaw (tối ưu)	HateXplain
RCNN	0.5689	0.6530 (+0.0841)	0.6399
Attention BiLSTM	0.6311	0.6344 (+0.0033)	0.6169
Transformer	0.4625	0.6274 (+0.1649)	0.6375

Bảng 3.4: Tổng hợp mức cải thiện F1 Macro

3.7.4 Thảo luận về hiệu suất của Transformer

Kết quả từ các thử nghiệm tối ưu hóa và đánh giá chéo cung cấp bằng chứng quan trọng để đánh giá giả thuyết rằng Transformer kém hơn RCNN và Attention BiLSTM do phụ thuộc vào bài toán.

Trên Jigsaw (đa nhãn), Transformer phiên bản gốc (Post-LN, mean pooling, 2 tầng) có F1 Macro 0.4625 - thấp hơn RCNN (0.5689) khoảng 0.1064 điểm. Tuy nhiên, sau khi áp dụng Pre-LN, CLS token và 4 tầng mã hóa, Transformer đạt F1 Macro 0.6274 - thu hẹp khoảng cách với RCNN (0.6530) xuống chỉ còn 0.0256 điểm (Bảng 3.2). Mức cải thiện 35.7% cho thấy vấn đề chính không phải do kiến trúc Transformer yếu kém, mà do cấu hình chưa phù hợp.

Trên HateXplain (đa lớp), Transformer đạt F1 Macro 0.6375, gần như tương đương với RCNN (0.6399) và vượt trội so với Attention BiLSTM (0.6169). Kết quả này càng củng cố nhận định rằng Transformer hoàn toàn có thể cạnh tranh với các mô hình dựa trên RNN khi được cấu hình đúng, đặc biệt trên bài toán đa lớp.

Hai yếu tố then chốt giúp cải thiện Transformer là:

- **Pre-LN:** LayerNorm được đặt trước sublayer thay vì sau residual connection, giúp gradient flow ổn định trong backpropagation, tránh gradient explosion ở các tầng gần đầu vào. Xiong và cộng sự (2020) [17] đã chứng minh Pre-LN vượt trội so với Post-LN, đặc biệt với Transformer nhiều tầng.
- **CLS token:** Token đặc biệt học cách tổng hợp toàn bộ câu thông qua cơ chế self-attention, thay vì mean pooling làm pha loãng tín hiệu từ các từ quan trọng. Phương pháp này được sử dụng phổ biến trong BERT [18].

Tuy nhiên, RCNN vẫn là mô hình có AUC cao nhất trên cả hai bộ dữ liệu (0.9846 trên Jigsaw, 0.8156 trên HateXplain), cho thấy kiến trúc kết hợp BiLSTM và max pooling là lựa chọn ổn định và đáng tin cậy cho bài toán phân loại văn bản độc hại ở nhiều điều kiện dữ liệu.

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đề tài đã hoàn thành việc nghiên cứu và cài đặt từ gốc các mô hình học sâu (Attention BiLSTM, RCNN, Transformer) để giải quyết bài toán phân loại bình luận độc hại, với các cải tiến sau:

Về kiến trúc mô hình:

- Transformer được cập nhật từ Post-LN sang Pre-LN, thêm learnable CLS token và tăng lên 4 encoder layers.
- RCNN được cải tiến thành EnhancedRCNN với thêm một lớp fc_hidden sau max pooling.
- Attention BiLSTM giữ nguyên kiến trúc gốc với self-attention pooling.

Về kỹ thuật huấn luyện:

- Masked Pooling: Loại bỏ padding tokens khỏi quá trình pooling, tránh pha loãng biểu diễn.
- Focal Loss: Giảm trọng số mẫu dễ, tập trung vào các mẫu khó (các lớp thiểu số).
- Label Smoothing: Làm mượt nhãn để cải thiện calibration, tránh overconfidence.
- Gradient Accumulation: Effective batch size 256 giúp gradient ổn định hơn.
- OneCycleLR: Tăng learning rate lên max rồi giảm về 0 trong một chu kỳ duy nhất.
- Early Stopping (patience = 3) dựa trên ROC-AUC để tránh overfitting.

Về đánh giá chéo: Mô hình được huấn luyện và đánh giá trên hai bộ dữ liệu: Jigsaw (159,571 mẫu, 6 nhãn) và HateXplain (20,109 mẫu, 3 nhãn). Kết quả cho phép so sánh tính nhất quán của các kiến trúc trên các bộ dữ liệu có đặc điểm khác nhau (Wikipedia vs. Twitter/Reddit).

Kết quả trên Jigsaw cho thấy RCNN đạt ROC-AUC cao nhất (0.9839 gốc, 0.9846 tối ưu) nhờ kết hợp BiLSTM và max pooling [1]. Attention BiLSTM đạt F1 Macro cao nhất ở phiên bản gốc (0.6311) nhờ cơ chế attention tập trung vào các từ quan trọng [2]. Transformer với các cải tiến Pre-LN + CLS token + 4 layers cho thấy mức cải thiện ấn tượng nhất: F1 Macro từ 0.4625 lên 0.6274 (tăng 35.7%).

Trên HateXplain, RCNN tiếp tục dẫn đầu với AUC 0.8156 và F1 Macro 0.6399. Transformer đạt F1 Macro 0.6375, gần tương đương RCNN và vượt Attention BiL-

STM (0.6169), cho thấy Transformer có thể cạnh tranh với RNN-based models trên bài toán multi-class khi được cấu hình đúng. Kết quả này bác bỏ giả thuyết cho rằng Transformer yếu kém một cách cố hữu so với RNN-based models - hiệu suất Transformer phụ thuộc nhiều vào cấu hình kiến trúc và đặc điểm bài toán.

Các kỹ thuật tối ưu hóa (masked pooling, focal loss, gradient accumulation, OneCycleLR) cải thiện hiệu suất ổn định trên tất cả mô hình. Thách thức chính vẫn là sự mất cân bằng dữ liệu: precision trên các lớp thiểu số (threat, severe_toxic, identity_hate) còn thấp (0.11-0.37), dẫn đến nhiều false positives.

Hướng phát triển bao gồm: sử dụng pretrained embeddings (GloVe, FastText), fine-tuning thêm các class weights, ensemble model kết hợp nhiều kiến trúc, data augmentation cho các lớp thiểu số, và hyperparameter tuning chi tiết hơn cho từng mô hình.

TÀI LIỆU THAM KHẢO

- [1] S. Lai, L. Xu, K. Liu, and J. Zhao, “Recurrent convolutional neural networks for text classification,” pp. 2267–2273, 2015. [Online]. Available: <https://www.aaai.org/ocs/index.php/AAAI/AAAI15/paper/view/9745>
- [2] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, “Hierarchical attention networks for document classification,” pp. 1480–1489, 2016. DOI: 10.18653/v1/N16-1174 [Online]. Available: <https://doi.org/10.18653/v1/N16-1174>
- [3] A. Vaswani et al., “Attention is all you need,” vol. 30, pp. 5998–6008, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
- [4] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” pp. 2980–2988, 2017. DOI: 10.1109/ICCV.2017.324 [Online]. Available: <https://doi.org/10.1109/ICCV.2017.324>
- [5] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826. DOI: 10.1109/CVPR.2016.308
- [6] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” 2017, arXiv:1711.05101. [Online]. Available: <https://arxiv.org/abs/1711.05101>
- [7] A. Graves, A. rahman Mohamed, and G. Hinton, “Speech recognition with deep recurrent neural networks,” pp. 6645–6649, 2013. DOI: 10.1109/ICASSP.2013.6638947 [Online]. Available: <https://doi.org/10.1109/ICASSP.2013.6638947>
- [8] L. N. Smith and N. Topin, “Super-convergence: Very fast training of neural networks using large learning rates,” in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, vol. 11006, SPIE, 2019, p. 1100612. DOI: 10.1117/12.2520589
- [9] Jigsaw and Google, “Jigsaw toxic comment classification challenge,” 2018. [Online]. Available: <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>

- [10] B. Mathew, P. Saha, S. M. Yimam, C. Biemann, P. Goyal, and A. Mukherjee, “Hatexplain: A benchmark dataset for explainable hate speech detection,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021. [Online]. Available: <https://hatexplain.github.io/>
- [11] R. Baeza-Yates and B. Ribeiro-Neto, “Modern information retrieval,” 1999.
- [12] I. Goodfellow, Y. Bengio, and A. Courville, “Deep learning,” 2016. [Online]. Available: <https://www.deeplearningbook.org/>
- [13] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735 [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [14] K. Cho et al., “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” pp. 1724–1734, 2014. DOI: 10.3115/v1/D14-1179 [Online]. Available: <https://doi.org/10.3115/v1/D14-1179>
- [15] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” *3rd International Conference on Learning Representations (ICLR)*, 2014, arXiv:1409.0473. [Online]. Available: <https://arxiv.org/abs/1409.0473>
- [16] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016. [Online]. Available: <https://arxiv.org/abs/1607.06450>
- [17] R. Xiong et al., “On layer normalization in the transformer architecture,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, vol. 119, 2020, pp. 10 524–10 533. [Online]. Available: <https://proceedings.mlr.press/v119/xiong20b.html>
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” vol. 1, pp. 4171–4186, 2019. DOI: 10.18653/v1/N19-1423 [Online]. Available: <https://doi.org/10.18653/v1/N19-1423>
- [19] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. DOI: 10.1016/j.ipm.2009.03.002 [Online]. Available: <https://doi.org/10.1016/j.ipm.2009.03.002>
- [20] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *3rd International Conference on Learning Representations (ICLR)*, 2015,

- arXiv:1412.6980. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: <http://jmlr.org/papers/v15/srivastava14a.html>
- [22] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, “Efficient BackPropagation,” pp. 9–50, 1998. DOI: 10.1007/3-540-49430-8_2
- [23] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. DOI: 10.1016/j.patrec.2005.10.010 [Online]. Available: <https://doi.org/10.1016/j.patrec.2005.10.010>
- [24] S. Raschka, “Model evaluation, model selection, and algorithm selection in machine learning,” *arXiv preprint arXiv:1811.12808*, 2018. [Online]. Available: <https://arxiv.org/abs/1811.12808>
- [25] D. H. Wolpert, “No free lunch theorems for optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997. DOI: 10.1109/4235.585893
- [26] N. Djuric, J. Zhou, R. Morris, M. Grbavic, V. Duk, and Bosch, “Hate speech detection with comment embeddings,” pp. 1–2, 2015. DOI: 10.1145/2740908.2742760 [Online]. Available: <https://doi.org/10.1145/2740908.2742760>
- [27] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” pp. 38–45, 2020. DOI: 10.18653/v1/2020.emnlp-demos.6 [Online]. Available: <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- [28] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of BERT: Smaller, faster, cheaper and lighter,” 2019, arXiv:1910.01108. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [29] J. Pennington, R. Socher, and C. D. Manning, “Glove: Global vectors for word representation,” pp. 1532–1543, 2014. DOI: 10.3115/v1/D14-1162 [Online]. Available: <https://doi.org/10.3115/v1/D14-1162>
- [30] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” 2013, arXiv:1301.3781. [Online]. Available: <https://arxiv.org/abs/1301.3781>